Wavelet's type system: monomorphic to the bone

Design doc — draft 1

Thesis. Every Wavelet function has a type expressible as a WIT function, and every Wavelet expression has a type expressible in WIT. The language therefore has **no polymorphism** — not at runtime, and not even inside the compiler's type language. Generic operations like `map` and `eq` do not disappear; they are **monomorphized**, existing once per concrete type. Reuse comes not from polymorphism but from three compile-time affordances — **deriving**, **functors**, and **overload resolution** — that generate and select among those monomorphic definitions for you.

1 Context

Wavelet is a small homoiconic language for the WebAssembly Component Model. Two prior commitments set the stage for this one:

- **Values are WIT values.** Wavelet has no data types of its own. Its booleans, strings, lists, records, variants, options, results, and flags *are* the WIT (WebAssembly Interface Types) types that cross component boundaries. There is no FFI and no marshalling layer: what you pass is what arrives.
- **One file is one component.** The unit you edit, compile, link, and deploy is a single Component-Model component, described by a WIT *world* the compiler synthesizes from your file.

This document extends that discipline from *values* to *types*. Just as Wavelet has no value a WIT signature cannot carry, it will have no **type** a WIT signature cannot name. The type system is, deliberately, exactly as expressive as WIT and not one inch more.

2 The decision

Two rules define the whole system:

1. **Every function's signature is a WIT function type.** Parameters and results are WIT types; the function is first-order and monomorphic. (It need not be *exported* — only exports appear in the synthesized `.wit` — but its signature must be *expressible* as one.)
2. **Every expression has a WIT type.** There is no `any`, no dynamic escape hatch, no boxed `interface{}`. Type-checking is total; an un-typeable expression is a compile error.

The immediate, deliberate consequence:

There is **no parametric polymorphism**. You cannot write `∀a. list<a> → u32`, because WIT cannot name it — WIT has no generic *functions* and no user-defined generic *types*. A "generic" operation is therefore not one function but a *family* of monomorphic functions, one per concrete type it is used at.

This is the world of C, Pascal, Ada, Zig, and Go-before-1.18: first-order and fully monomorphic. The bet of this design is that Wavelet can keep that world's simplicity and WIT-faithfulness while avoiding its historical ergonomic potholes — by making the per-type generation and selection of code a *first-class, in-language* activity rather than an out-of-band chore.

2.1 What WIT can and cannot express

The rules above are a direct shadow of WIT's own limits. WIT can name:

- primitives `bool`, `u8..u64`, `s8..s64`, `f32`, `f64`, `char`, `string`;
- four built-in generic *constructors* — `list<t>`, `option<t>`, `result<t,e>`, `tuple<...>`;

- user-defined *nominal* types — `record`, `variant`, `enum`, `flags`, `resource`;
- aliases `type x = y`.

And WIT *cannot* name:

- **generic functions** — `func(list<t>) -> u32` is illegal; every WIT function is monomorphic. (This is why even `len`, let alone `map`, has no single WIT type.)
- **user-defined generic types** — there is no `type stack<t>`.
- **recursive types** — no `type tree = node(list<tree>)`. Self-referential data is encoded as an *arena* (a flat node table plus a root index), exactly as Wavelet's own "code as data" `tree` type already is.
- **first-class function types** — a callback is a single-method `resource`.

Because the type system is the shadow of this list, the same four walls bound Wavelet: no generic functions, no generic types, no recursive types (arena-encode them), and functions-as-values become resources at boundaries.

3 The one distinction that makes it ergonomic

The design lives or dies on keeping two ideas apart that are easily conflated:

Parametric polymorphism — rejected	**Ad-hoc overloading** — embraced
One function, *all* types: `∀a. (a,a)→bool`.	*Many* monomorphic functions, *one* name.
Not a WIT type. Would smuggle a non-WIT type into the compiler.	Each function is a plain WIT function. The name is resolved away at compile time.
`eq` is a single generic definition.	`eq` is an *overload set*: `eq-u32`, `eq-string`, `eq-point`, …
Resolved by *instantiation at runtime* (boxing / dictionaries).	Resolved by *the checker at each call site* from static argument types.

Everything below is two motions on the right-hand column: *generate the many functions cheaply*, and *let one name select among them.*

4 Reuse without polymorphism: three affordances

4.1 Deriving — kill the per-type boilerplate

The operations that "should be generic" — equality, ordering, hashing, stringification — are *structural*: their bodies follow the shape of the type. Because in Wavelet a type *is* ordinary WAVE data (`DefType` is just a form), a **derive macro** is literally a compile-time function from a type-form to a list of definition-forms. It walks the type and emits the monomorphic operations.

```
DefType point {x: s32 y: s32}

Derive {Eq Ord Show} point      // proposed syntax: emits, for `point`,
                                //   eq-point, compare-point, show-point
Def same Fn {a: point b: point}
  eq(a b)                       // overload-resolves to the derived eq-point
```

This is Rust's `#[derive(Eq, Ord)]` and Haskell's `deriving (Eq, Show)` — with one difference that matters here. Haskell's derived `==` has type `Eq a => a -> a -> Bool`: still polymorphic, resolved by a runtime dictionary. Wavelet's derived `eq-point` is a concrete `(point, point) -> bool` selected statically. Same ergonomics at the keystroke; nothing polymorphic survives to runtime.

4.2 Functors — generic data structures as compile-time component instantiation

A container like `Set` cannot be a generic *type*. But it can be a **functor**: a component parameterized over its element type and that type's operations, *instantiated once per element type at compile time*. After instantiation, everything is concrete and monomorphic.

This is the ML module system's central idea, and it maps onto the Component Model with no slack:

ML module system	**Wavelet / Component Model**
signature (module type)	WIT *interface*
structure (module)	*component*
functor `Make(M: ORDERED): SET`	compile-time function from component to component
`module StringSet = Make(String)`	parameterized `Import` (below)

Crucially, Wavelet *already has the machinery*: its macro system instantiates components at compile time. A functor is just a component you import *with a type argument*, producing qualified names the way every other import does:

```
// proposed syntax: instantiate the Set functor at two element types
Import {pkg: "wavelet:coll/set" elem: string as: strs}
Import {pkg: "wavelet:coll/set" elem: point  as: pts}

Def demo Fn {}
  Let {s: strs/new()}
    Do [ strs/add(s "hello")
         strs/contains(s "hello") ]   // → true, fully statically typed
```

Each `Import` stamps out a monomorphic `Set` specialized to its `elem`, with its own concrete WIT interface (§7). The element type's required operations (e.g. `eq`, `compare`) are supplied by the same overload-resolution that the call sites use, so instantiating `Set` for a `point` "just works" once `point` derives `Ord`.

4.3 Overload resolution — make the call sites pleasant

Generation gives you `eq-u32`, `eq-string`, `eq-point`. Selection lets you write `eq(a b)` and have the checker pick the right one from the static types of `a` and `b`. An **overload set** is simply several monomorphic bindings that share a name; resolution is per-call-site and entirely compile-time. Rules:

- Resolve on the static argument types; on ambiguity, *error at the call site*, fixable by qualifying (`json/eq`).
- Where arguments don't determine it (return-type-directed cases such as `read`), the expected type from a `The` ascription or the surrounding context decides.
- Importing two components that both define `eq` for `string` is *not* a global conflict — the overload set is just unioned, and only an *actually ambiguous call* is an error. There is no "one canonical instance per type" rule to enforce, because there are no polymorphic functions whose dictionaries must agree.

That last point is worth dwelling on: this is **typeclasses with the polymorphism removed**. You keep the per-type-instance organization for *defining* operations and the by-name dispatch for *calling* them, but you never form a constrained polymorphic function (`Eq a => …`) — which is the one construct that would reintroduce a non-WIT type. The coherence headaches that haunt typeclass systems never arise.

5 Inference and literals

Inference is **bidirectional and monomorphic**. The checker infers concrete WIT types for unannotated locals and propagates expected types inward (which is how overloads resolve), but there are no type *schemes* to generalize — there is nothing to generalize *over*. Annotations are needed only to disambiguate return-type-directed overloads, via the `The` ascription form.

Numeric literals are **context-resolved**, not polymorphic. `42` is a literal whose type is fixed at its single use site by the expected type — `u8` here, `s64` there — defaulting to `s64` (and float literals to `f64`) when unconstrained, with a compile-time range check. A literal is a syntactic token, not a function, so resolving it per use introduces no `∀`; this is exactly how C, Ada, and Rust type literals without being "polymorphic."

6 Core language vs. standard library

Wavelet's guiding principle is a minimal core with everything else delivered as components. The type system honors it: **the core provides only the typing discipline and the compile-time metaprogramming substrate; the standard library provides the generic-feeling vocabulary by using that substrate.** None of the "reuse" affordances are core — they are ordinary library code built on core machinery.

The dividing line has a sharp test: **anything that must run during type checking is core; anything that can be expressed as a macro (a compile-time `tree → tree` component) or as a family of ordinary monomorphic definitions is standard library.**

Mechanism	Layer	Why
The two typing rules + total checking	core	The definition of the language's static semantics.
Bidirectional monomorphic inference	core	Part of the checker; nothing to delegate it to.
Overload resolution — selecting among same-named monomorphic functions by static argument/expected type	core	Inherently a type-checker activity: it needs the static types, which macros (running before checking) do not have. This is the one piece that *cannot* be a library.
Literal context-resolution + defaulting (`s64` / `f64`)	core	Decided during checking from expected types.
`The` ascription, `DefType`	core	Already special forms; they feed the checker.
Macro system + compile-time component instantiation (the expander)	core	The substrate every affordance is built on.
WIT-world synthesis + export name-mangling	core	The compiler owns the boundary it emits.
`Derive` and the derivers `Eq` / `Ord` / `Show` / `Hash`	stdlib	Each is a macro — a `tree → tree` component. Users can ship their own.
Functor components (`Set`, `Map`, `sort`, …) and the instantiation convention	stdlib	Ordinary components, specialized via the core macro/instantiation substrate.
Overload sets `eq`, `compare`, `show`, `add`, `map`, `fold`, `each`, …	stdlib	Families of monomorphic definitions — usually *emitted* by derive/functor macros.

Mechanism	Layer	Why
User-authored derivers and functors	user	Just more macros and components; no privileged status.

Two clarifications this table compresses:

- **Overloading is core, but no overloaded name is.** The *rule* that a name may stand for several monomorphic functions and be resolved by the checker lives in the core. The *name* `eq` — and the fact it has a `string` and a `point` variant — is entirely standard-library content. Add a type, derive `Eq`, and the overload set grows; the core never changed.
- **Two flavors of functor.** A *source functor* is purely a macro: it takes an element type-form and splices out specialized `DefType` / `Def` forms — no core support beyond the existing expander. A *binary functor* (a precompiled, perhaps Rust-authored, parameterized component) instead needs the compiler to substitute the element type into the component's WIT and monomorphize it; that specialization pass is the one genuinely new piece of *core* functor support. The standard library can ship most generic structures as source functors and lean on the core only for the binary case.

The upshot: the core's seventeen special forms gain *no* new members for generics. They gain a type checker (with overload resolution and literal defaulting), and the rest — `Derive`, `Set`, `eq`, `map` — rides in as components, exactly like the macros and standard library that already do.

7 Worked example, with the WIT it produces

A `point`, equality derived for it, and a `Set` of points — and the `.wit` the compiler synthesizes. The headline is that **nothing in the WIT is generic**: every synthesized signature is a concrete, ordinary WIT function or type.

7.1 Source

```
Package "demo:geo@0.1.0"

DefType point {x: s32 y: s32}
Derive {Eq Ord Show} point

Import {pkg: "wavelet:coll/set" elem: point as: pts}

Export nearest-set
Def nearest-set Fn {ps: list<point>}
  Let {s: pts/new()}
    Do [ each(ps Fn {p} pts/add(s p))   // each is itself monomorphic here
         s ]
```

7.2 Synthesized WIT

A derived, *exported* `eq` / `show` and the instantiated `Set` become concrete interfaces. Note the name-mangling: WIT has no overloading, so the overload set `eq` lowers to distinctly named functions.

```
package demo:geo@0.1.0;

interface types {
  record point { x: s32, y: s32 }
}

interface compare {
  use types.{point};
```

```
  eq-point: func(a: point, b: point) -> bool;
  compare-point: func(a: point, b: point) -> s32;
  show-point: func(v: point) -> string;
}

// the Set functor, instantiated at `point`: a monomorphic resource interface
interface point-set {
  use types.{point};
  resource set {
    constructor();
    add: func(value: point);
    contains: func(value: point) -> bool;
    size: func() -> u32;
  }
}

world geo {
  export types;
  export compare;
  export point-set;
  export nearest-set: func(ps: list<point>) -> point-set.set;
}
```

Compare what a *generic* set would have wanted — `resource set<t>` with `add: func(value: t)` — which WIT simply cannot write. The functor produces `point-set` instead, and a second instantiation would produce `string-set`, each a separate, concrete interface.

8 Comparison with other languages

8.1 Go before 1.18 — the cautionary tale Wavelet must beat

Go shipped for a decade as a first-order, monomorphic language with *no* generation or overloading affordances. To reuse a container you had two bad options:

```
// Option A: hand-write (or code-gen) one type per element type.
type StringSet map[string]struct{}
func (s StringSet) Add(v string)           { s[v] = struct{}{} }
func (s StringSet) Contains(v string) bool { _, ok := s[v]; return ok }
// ... now write IntSet, PointSet, ... all over again.

// Option B: erase the type with interface{} — and lose static safety.
type Set map[interface{}]struct{}   // boxes every element; runtime assertions
```

Option A meant copy-paste or an *out-of-band* code generator (`go generate`, genny) whose output drifted from your types and whose call sites were verbose. Option B threw away the very static typing that justifies the language, and paid for it with boxing and runtime type assertions. The pain was real enough that Go added parametric generics in 1.18.

Wavelet inhabits the *same monomorphic world* as pre-1.18 Go — and explicitly keeps Option B off the table (there is no `interface{}`; the nearest thing, the `tree` arena type, is itself a perfectly ordinary WIT type, not an untyped escape hatch). What Wavelet adds is precisely the two layers Go lacked:

Go ≤ 1.17 friction	**Wavelet's answer**
Code-gen is out-of-band (`go generate`); output drifts.	Generation is *in-language*: derive macros + functor `Import`, run by the compiler.

Go ≤ 1.17 friction	Wavelet's answer
`interface{}` boxes and defeats the type checker.	No `any`; every expression keeps a concrete WIT type.
Call sites verbose (`StringSet`, `IntSet`, distinct names).	Overload sets: write `add(s x)` / `eq(a b)`, resolved statically.
`sort.Interface` boilerplate per type.	`Derive {Ord} t` emits it; `sort` is a functor instantiated at `t`.

The wager is that derive + functors + overloading make the monomorphic world *comfortable enough* that Wavelet never feels Go's pull toward generics — a pull that, for Wavelet, would mean expressing a type WIT cannot name.

8.2 Ada — the closest precedent

Ada is the mature language built on exactly these pieces: **generic packages** you *explicitly instantiate per type*, plus strong **static overload resolution** (on argument types *and* return type). `package Int_Sets is new Sets(Integer);` is an ML functor application in Ada's clothing, and `Put` resolving across a dozen overloads is Wavelet's `eq` / `show` dispatch. Ada demonstrates the combination scales to large, long-lived systems without parametric polymorphism in the modern sense.

8.3 Zig — generics as compile-time code, no overloading

Zig's `comptime` makes "generics" ordinary functions that run at compile time and *return types*; everything monomorphizes and no runtime polymorphism remains — the same end state as a Wavelet functor instantiation. Instructively, Zig deliberately has *no* function overloading, which makes its call sites carry the type in the name or the namespace. Zig is the proof that compile-time monomorphization is enough for real systems; Wavelet's overload layer is the ergonomic piece Zig chooses to forgo.

8.4 OCaml / SML functors, MLton — reuse without runtime polymorphism

ML's module layer is where "abstraction without core polymorphism" was worked out: functors parameterize structures over signatures, and a whole-program compiler like MLton *monomorphizes and defunctorizes* the result into code with no polymorphism left. That is precisely Wavelet's compilation story — except Wavelet pushes the discipline up into the surface language so the *source*, not just the object code, is monomorphic. (Haskell's *Backpack* lifts the same functor idea to the package level, which is even closer to "components parameterized by interface".)

8.5 Rust / Haskell `deriving` — the model for derivation

Both languages' `deriving` / `derive` is the direct inspiration for Wavelet's derive macros: walk a type's structure, emit the obvious operations. The distinction, restated: in Rust and Haskell the derived operation is *parametric* (`impl<T: Eq> …`, `Eq a =>`), resolved by monomorphization or dictionaries; Wavelet's derived operation is a *concrete* function joined to an overload set. Wavelet borrows the ergonomics and drops the polymorphism.

9 Costs and risks (honest)

- **Code-size blowup.** Every instantiation is real code. This is monomorphization's universal tax (Rust, C++, Zig pay it too) and the price of WIT-faithfulness.
- **Instantiate-before-use.** A `Set` for `point` must be imported/instantiated before use, and a derive must precede the call that resolves to it — consistent with Wavelet's existing "macros must be in scope before use" reader rule.

- **No abstraction over unknown types.** You cannot write a function generic over a type it does not know at compile time; every "generic" use must resolve to a concrete type at a known site. For a component language whose boundaries are monomorphic WIT anyway, this bites far less than in a general-purpose language — the pain is confined to internal reuse, which is exactly what the three affordances target.
- **Overload ambiguity.** Resolution must be specified tightly enough to stay predictable; the fallback is always "error, then qualify the name."
- **The Go lesson.** If the affordances are *not* comfortable enough, users feel the pull toward real generics. Keeping derive/functor/overload ergonomic is not polish — it is what makes the whole position tenable.

10 Open questions

1. **Functor spelling.** Is the parameterized `Import {pkg: … elem: t as: …}` the right surface, or should functor application be its own form (e.g. a `Instantiate` macro)? How are functors with *several* type/operation parameters expressed?

2. **Derive surface and extent.** Is `Derive {Eq Ord Show} t` right? Which classes ship built-in (Eq, Ord, Show, Hash, …), and can users author their own derivers (a deriver is, after all, just a `tree → tree` component)?

3. **Overload declaration.** Do same-named `Def`s with distinct argument types *implicitly* form an overload set, or is an explicit grouping form required? What exactly is the resolution algorithm (argument-only, or full bidirectional with return-type), and how does it interact with `The`?

4. **Name mangling at the boundary.** When an overload set is *exported*, what are the WIT names (`eq-point`, `compare-point`, …) — compiler-chosen, or user-controlled via `Export`?

5. **Resources vs. arenas for recursion.** Recursive data is arena-encoded by rule; should the language offer a built-in derive that generates the arena representation (and its accessors) from a recursive *type description*, so users don't hand-roll arenas the way the `tree` type currently is?